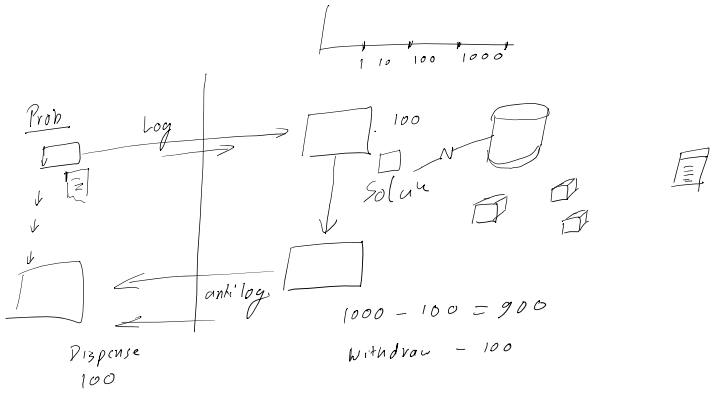
Type
Range
Lifetime
Scope





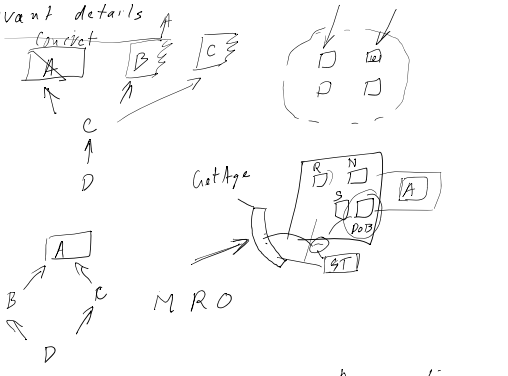
Object Oriented

Major / Minor pillars
Inheritance
Polymorphism
encapsulation
abstraction
object
class

Paradigm

Major Pillars [4]

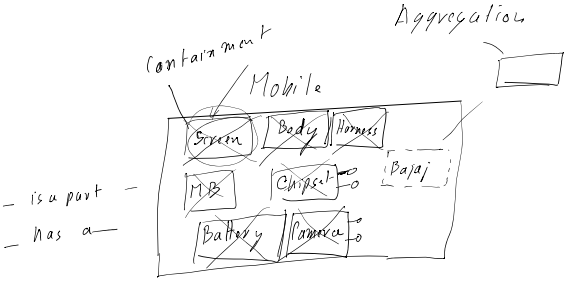
1) Abstraction — Floating relevant details
2) Encapsulation — Data hiding / Interface
3) Modularisation →
   - Composition
     - Aggregation
     - Containment
   - Inheritance
     - Single
     - Multiple
     - Multilevel
     - Hybrid
4) Polymorphism



User Conv.

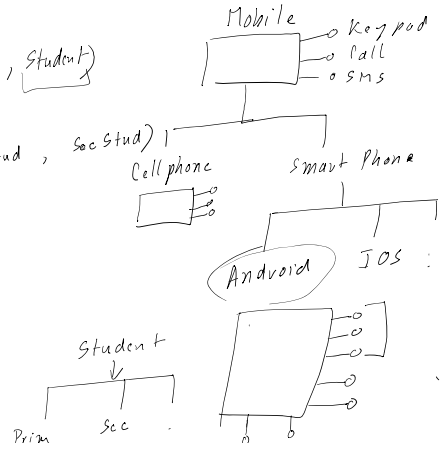
Student

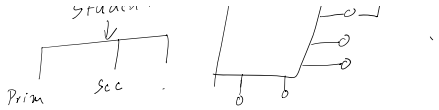
Roll no / ID
Name
Age
Address
Gender
Std
Sec
Class teacher
School Name
o Allegiries
o Blood grp.
Fav Cartoon
Fav Flavour
Marks

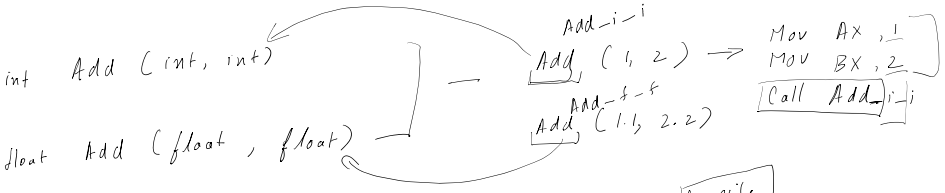


Group by (Student, Student)

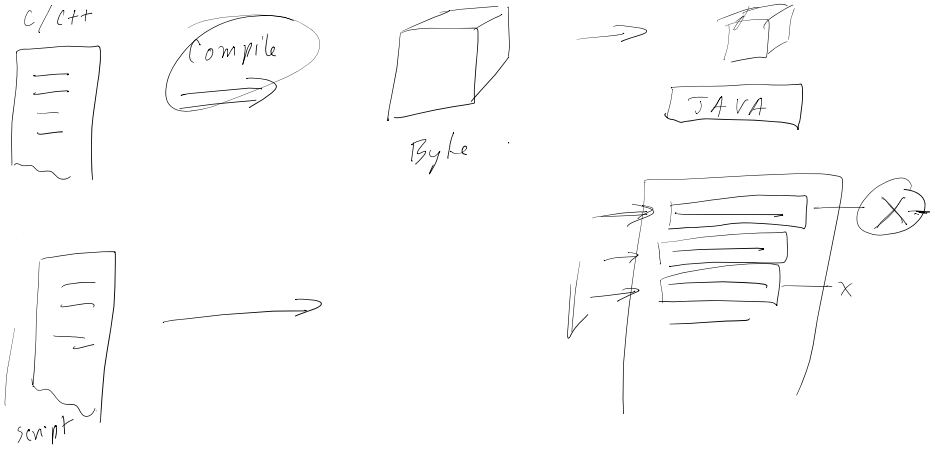
Groupby (Stud, PStud, SecStud)

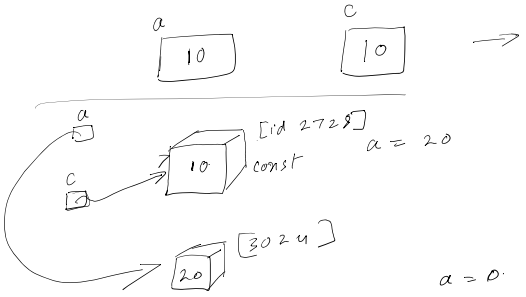


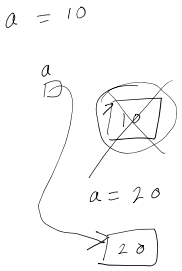

Prim
Scc

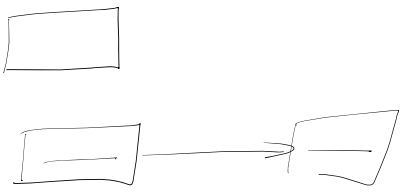

int Add (int, int)
float Add (float, float)
Add_i_i
Add (1, 2)
Add_f_f
Add (1.1, 2.2)
Mov AX, 1
MOV BX, 2
Call Add_i_i


Compile
Runtime
Call Add_i_i
Add_f_f


C/C++
Compile
Byte
JAVA
X
X
script


a
10
c
10
a
[id 2728]
10
const
a = 20
c
[3024]
20
a = 0
[a += 1]


a = 10
a
10
a = 20
20



8	4	2	1	
0	0	0	0	→ 0
0	0	0	1	→ 1

① 0 1 0 → 10

IEEE

1 0 1 1 → 11

↳ 10.5 → 105 × 10^{-1}

0 × 10^{0}

float →

included Start / End — excluded

S1[3 : 5]

Start End

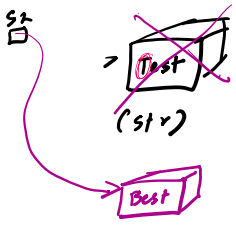
H

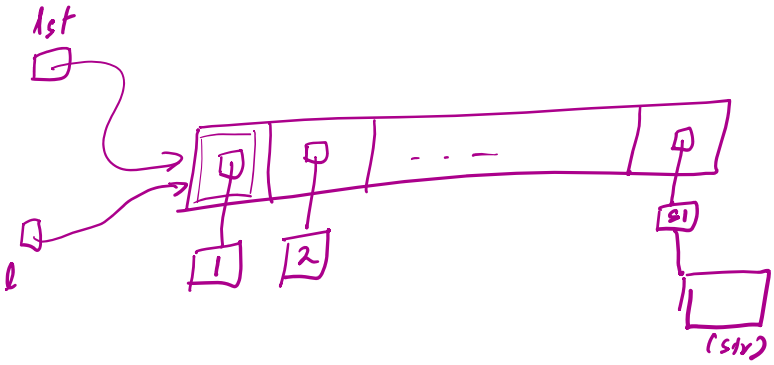
E S

S

T E

E





Mod List (lst)

def ModList (l)

__repr__()

__str__() { }

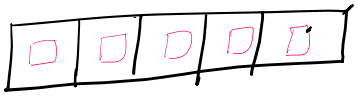
print (loan1)

print (loan1.__repr__())

print (repr (loan1))

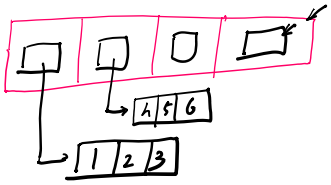
Value
int

Key



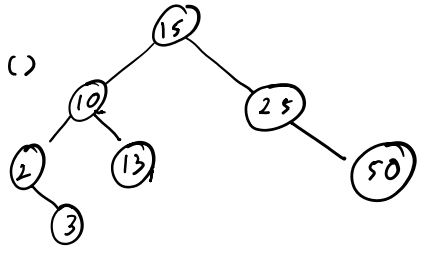
lst = [(1,2,3), (4,5,6)]

lst[0][1]

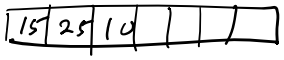


len (obj)

obj.__len__()



L/T



sort

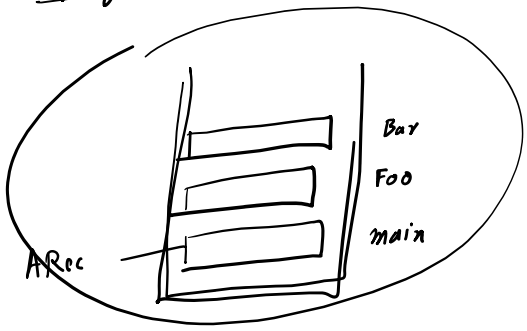
	Mutables		Immutables
			Number
			strings
ordered Heterogenous	List	——	Tuples
Unordered Keys	Sets	—	Frozen Sets
Unordered K-V pair	Dict	-	Named Tuples

Minor Pillars -

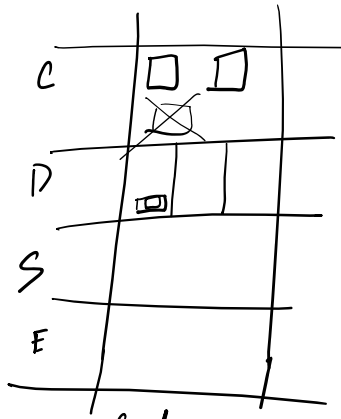
1) Persistence → across time & space
2) Concurrency → Language

concurrent vs. simultaneous

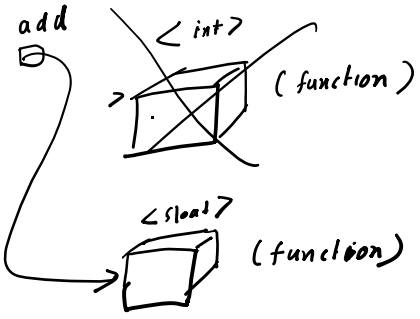
3) Typing System - Dynamic



a = 10
a = 20



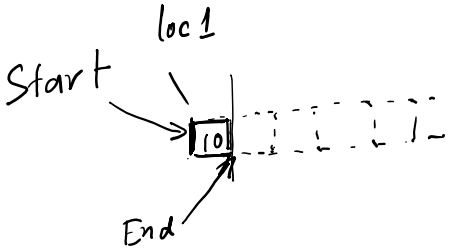
CS - Code
DS - Data
ES - Extra
SS - Stack

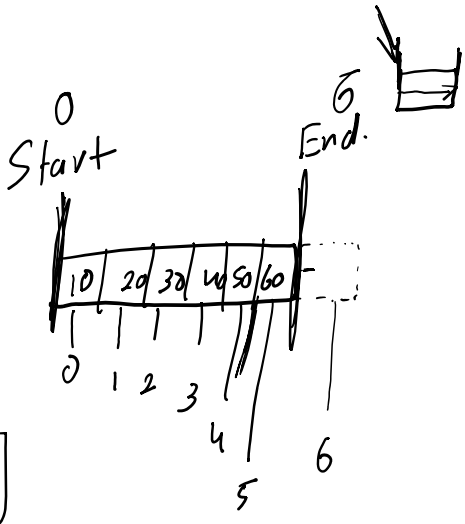


Bash [Linux] → Bash script → .sh

Cmd [Win] → Batch script → .bat

Powershell [Win] → PowerShell script → .ps1





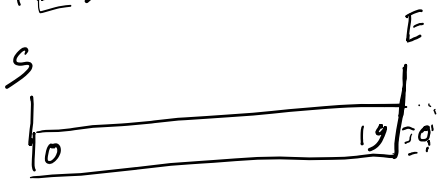
[Start End]

Exclude

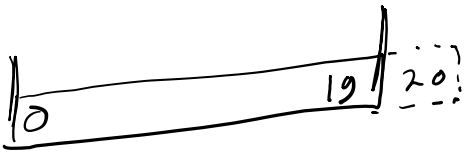
Start End

Include Exclude

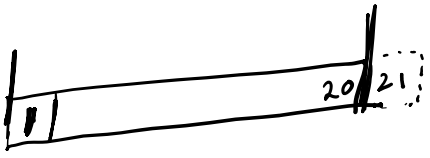
range (20)

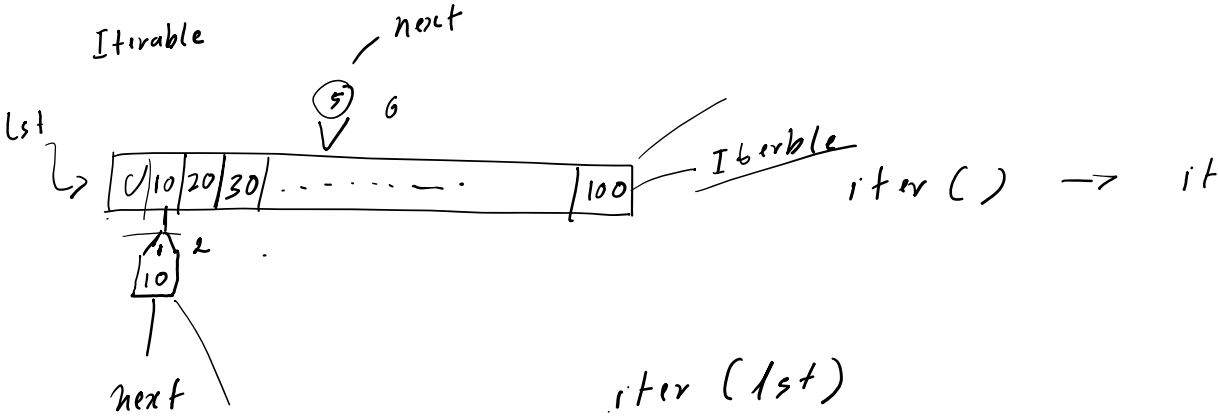


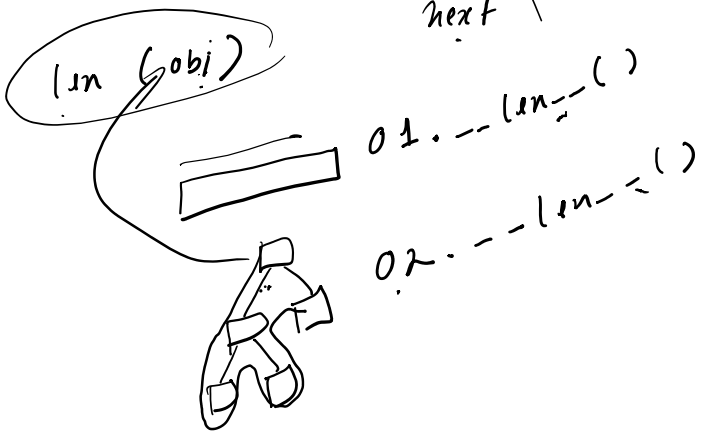
range (0, 20)



range (1, 21)





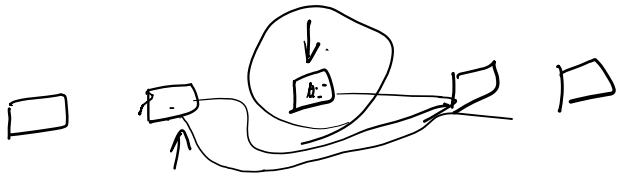


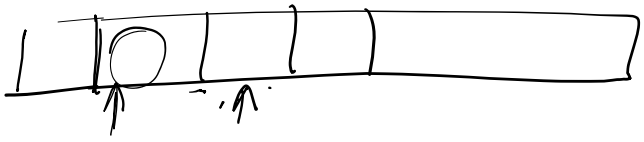
iter(lst)

lst.__iter__()

ıralor



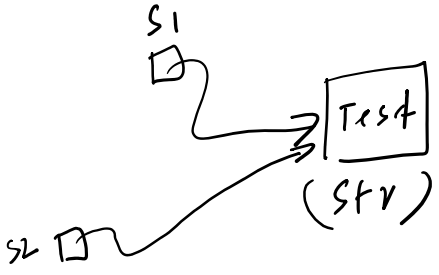


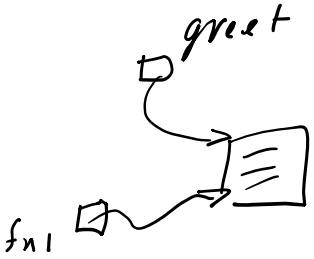


Consumer

My Math__init__

algebra

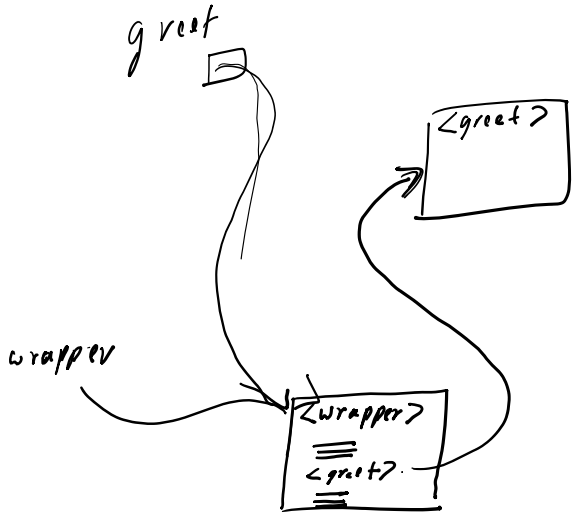




fn1 = greet

S2 = S1

greet



greet = logger (greet

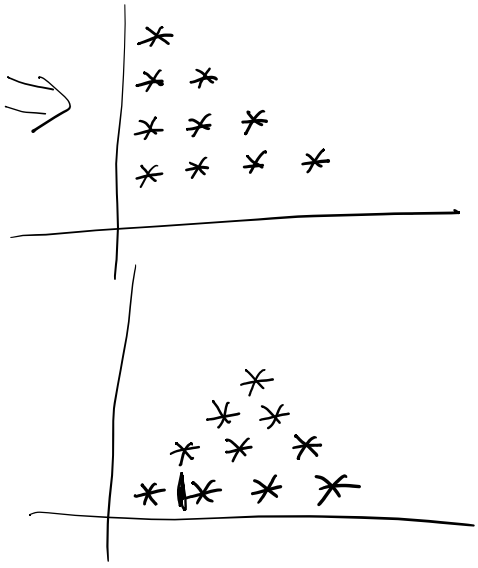
def logger (<greet> → fn

def wrapper (

<greet> → Fn

greet ()

n = 4



n = 4